MissileSim 3D: Learning Autonomous Missile Navigation through Procedurally Generated Tunnels with Deep RL

Darsh Choudhary (B24CM1019) Vinamra Gupta (B24CM1068)
GitHub: https://github.com/Vinamra3215/Missile-Sim

1 Problem

An agent must fly a missile through a 3D tunnel filled with static pipes, *moving* pipes and *gates* (walls with a single hole), and strike a target core at the far end, using only onboard range sensors. The setting mirrors real autonomous navigation (drones, UAVs, guided vehicles): partial observability, moving obstacles and sparse terminal outcomes. To force *generalisation* rather than route memorisation, every episode is a newly generated map, and all results are measured on held-out maps never seen in training [7].

2 Environment

The simulator is written in PyTorch as a *batched* environment: N independent missiles (128 for PPO, 64 for DQN) are stepped together as tensors, including collision checks and ray casting, which gives ∼5k steps/s on one CPU core. **World.** A square tunnel $|y|, |z| \le 8$ along $+x$ of length L, target sphere (radius 1.8) at $x{=}L$ at a random height/side. Obstacle slots are spaced ≥9 units apart; each is a pipe (infinite cylinder, random angle in the yz-plane, offset, tilt and radius 0.8–2.6) or a gate (thickness 0.6, circular hole at a random position). Moving pipes oscillate: $\mathbf{p}(t) = \mathbf{p}_0 + \mathbf{m}\, A \sin(\omega t + \phi)$. **Dynamics** (arcade flight, $\Delta t{=}0.05\,\mathrm{s}$): pitch θ, yaw ψ and speed v are updated by the action, then the missile moves along its heading $\mathbf{f}$:

$$\theta \leftarrow \mathrm{clip}(\theta + 1.8\Delta t\, u_\theta, \pm 1.2), \quad \psi \leftarrow \psi + 1.8\Delta t\, u_\psi,$$
$$v \leftarrow \mathrm{clip}(v + 12\Delta t\, u_v,\ 8,\ 22), \quad \mathbf{p} \leftarrow \mathbf{p} + v\,\mathbf{f}(\theta,\psi)\,\Delta t.$$

The missile can never stop ($v \ge 8$), so hovering is impossible. **Curriculum.** A difficulty $\ell \in [0, 2]$ scales tunnel length (60→220→400), obstacle count (2→16→30), pipe size and tilt, fraction and speed of moving pipes (from $\ell{=}0.4$), gate frequency (from $\ell{=}0.3$) and gate hole radius (4.5→2.0); $\ell \in [1, 2]$ is an *expert tier*. Maps are sampled at $\ell' \sim U[\ell - 0.3, \ell]$ and ℓ rises by 0.1 once the success rate over the last ∼500 episodes near the top level reaches 75% [6].

Implementation note. The simulator (map generation, kinematics, collision detection and ray-cast sensing) is custom code written for this project with AI assistance, not an existing physics engine; it serves as the testbed, and the focus of our work is the RL side: problem formulation, reward and curriculum design, and training and evaluating PPO and DQN. Details are in the repository README.

3 RL Formulation

State space $\mathcal{S} \subset \mathbb{R}^{61}$ (continuous, normalised): a 5×5 grid of ray-cast distances in a 70° cone ahead of the nose (range 40, scaled to $[0, 1]$) at time t *and* $t{-}1$, so that obstacle motion is observable (50); the unit vector to the target in the missile's body frame (3); distance to target (1); speed (1); pitch (1); $\sin\psi, \cos\psi$ (2); position in the cross-section $y/8, z/8$ (2); remaining tunnel length (1). **Action space** $\mathcal{A}$ (discrete, $|\mathcal{A}|{=}7$): coast, pitch up, pitch down, yaw left, yaw right, boost, brake (i.e. $u_\theta, u_\psi, u_v \in \{-1, 0, 1\}$, one at a time). **Reward.** With d_t the distance to the target,

$$r_t = 0.1\,(d_{t-1} - d_t) - 0.01 + 20\mathbb{1}_{\mathrm{hit}} - 10\mathbb{1}_{\mathrm{crash}} - 5\mathbb{1}_{\mathrm{timeout}}.$$

The dense progress term guides learning, the step penalty discourages dawdling, and the terminal terms encode the true objective. A *hit* is $\|\mathbf{p} - \mathbf{p}_{\mathrm{tgt}}\| < 2.2$; a *crash* is contact with a wall, pipe or gate; a *timeout* occurs after $\lceil 1.8(L{+}6)/(8\Delta t) \rceil$ steps. Timeouts are treated as truncations (bootstrapped), crashes and hits as true terminals.

4 Algorithms and Baselines

Both agents are implemented directly in PyTorch, without RL libraries. **PPO** [2] (main): separate actor and critic MLPs (2×256, ReLU, orthogonal init.), clipped surrogate objective $L = \mathbb{E}\big[\min(\rho_t A_t, \mathrm{clip}(\rho_t, 0.8, 1.2)A_t)\big]$ with GAE [3] ($\gamma{=}0.995, \lambda{=}0.95$), entropy bonus 0.01, value loss 0.5, rollouts of 32×128 steps, 4 epochs × 4 minibatches, Adam $3{\cdot}10^{-4}$ with linear annealing, gradient clip 0.5. **DQN** [1] (baseline): Double DQN [4] with a dueling head [5], 3-step returns, Huber loss, replay buffer $3{\cdot}10^5$, batch 256, soft target update $\tau{=}0.005$, ϵ: $1 \to 0.03$ over the first 20% of training, $\gamma{=}0.99$, Adam $3{\cdot}10^{-4}$. **Baselines:** a uniform *random* policy, and a hand-coded *steer-to-target* rule that turns toward the target and ignores obstacles (used only for comparison). **Protocol.** Each algorithm: 3 seeds × $3{\cdot}10^6$ environment steps, with identical curriculum (PPO ≈11 min, DQN ≈16 min per run on a desktop CPU). Evaluation: greedy policy, 300 held-out maps per level (separate seed), identical maps for all policies.

5 Results

Table 1: Success rate (%) on unseen maps, mean ± std over 3 seeds.

Policy	$\ell{=}0$	0.5	1.0	1.5	2.0
Random	1.0	0.0	0.0	0.0	0.0
Steer rule	49.0	3.3	0.0	0.0	0.0
DQN	95.1±1.9	88.9±2.6	47.0±3.8	8.1±2.7	0.4±0.5
PPO	**99.5**±0.4	**94.8**±0.4	**85.5**±0.7	**55.3**±4.6	**14.2**±3.0

Both RL agents learn to avoid obstacles far beyond the steer-to-target rule, which succeeds on 49% of the easiest maps but almost never once gates and moving pipes appear: obstacle avoidance is essential and is learned, not hand-coded. PPO is better at every level and climbs the curriculum faster (level 1.2 vs. 0.87 after $3{\cdot}10^6$ steps), and the gap widens with difficulty (85.5% vs. 47.0% at $\ell{=}1$; 55.3% vs. 8.1% at $\ell{=}1.5$). PPO also generalises into the expert tier beyond the level it trained on (14% at $\ell{=}2$). Seed-to-seed variance is small for both methods, so the conclusions are robust.

6 Discussion

PPO's advantage grows with difficulty. Plausible explanations (not tested separately here) are its stable on-policy updates over large batches from many parallel environ-

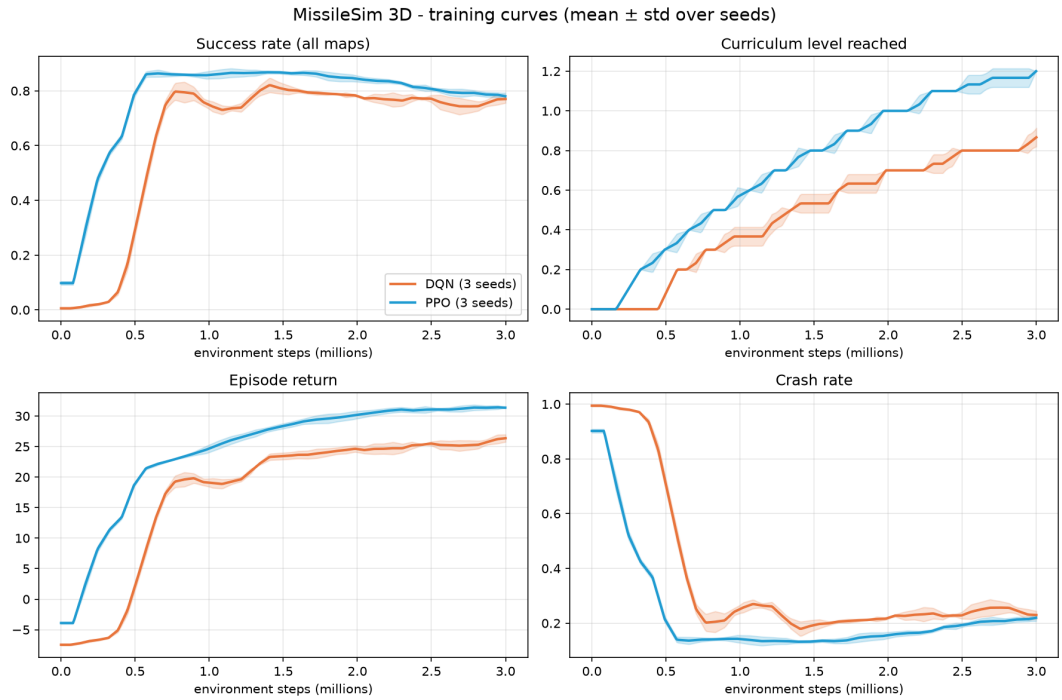


Figure 1: Training curves (mean ± std over 3 seeds). Success and crash rates are measured on the maps of the current curriculum level, so they dip as the curriculum makes maps harder.

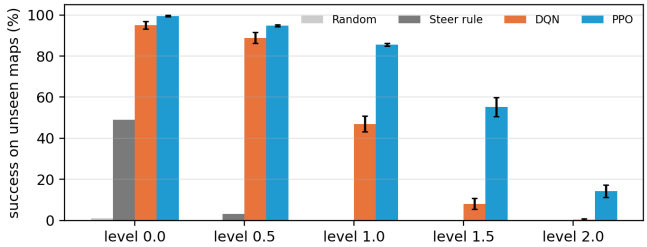


Figure 2: Held-out evaluation by difficulty level (error bars: std over seeds).



Figure 3: The trained PPO agent in the 3D viewer on unseen maps: dodging moving pipes (striped), threading a gate hole, and approaching the target core.

ments, and that DQN's replay buffer keeps mixing in transitions from earlier, easier curriculum levels, so its value estimates lag behind the changing map distribution. **Limitations:** simplified arcade dynamics (no aerodynamics or roll), discrete actions, and a fixed training budget; the expert tier remains unsolved, suggesting longer training, recurrent policies or continuous control as future work. A 3D viewer (Ursina) renders trained policies and supports human play for comparison.

References

[1] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, 518, 2015.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[3] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *ICLR*, 2016.

[4] H. van Hasselt, A. Guez, D. Silver, "Deep reinforcement learning with double Q-learning," *AAAI*, 2016.

[5] Z. Wang et al., "Dueling network architectures for deep reinforcement learning," *ICML*, 2016.

[6] Y. Bengio, J. Louradour, R. Collobert, J. Weston, "Curriculum learning," *ICML*, 2009.

[7] K. Cobbe, O. Klimov, C. Hesse, T. Kim, J. Schulman, "Quantifying generalization in reinforcement learning," *ICML*, 2019.